

VEILLE TECHNOLOGIQUE — 420-1SH-SW

TicketFlow ITSM

Automatisation du triage des incidents informatiques par IA

Flask

Gemini 2.5 Flash

RAG

SQLite

Bootstrap 5

Gabriel Racine

Contexte et but du projet

Le problème

- Les techniciens ITSM reçoivent des centaines de billets par jour
- Chaque billet doit être lu, catégorisé et priorisé manuellement
- Les délais de triage ralentissent la résolution des incidents
- Le risque d'erreur humaine est élevé sous pression

La solution — TicketFlow

- API REST Flask exposant un endpoint de triage automatique
- IA (Gemini 2.5 Flash) pour classer et prioriser chaque billet
- Moteur RAG pour enrichir l'IA avec des procédures internes
- Dashboard technicien + portail demandeur en Bootstrap 5

Objectif : Réduire à zéro le triage manuel en automatisant la classification et la priorisation des incidents ITSM.

Technologies utilisées

Flask 3.1

Framework API REST

Blueprints, routes JSON versionnées
(/api/v1/)

Google Gemini 2.5 Flash

Moteur IA principal

Sortie JSON structurée (Pydantic),
embeddings vectoriels

Groq / Llama 3

Fallback IA

Triage de secours si quota Gemini atteint
(429)

ChromaDB

Base vectorielle

Persistence des embeddings ITSM sur
disque

SQLite + SQLAlchemy

Persistence

Modèles Ticket, TriageResult, RagHistory
+ migrations Alembic

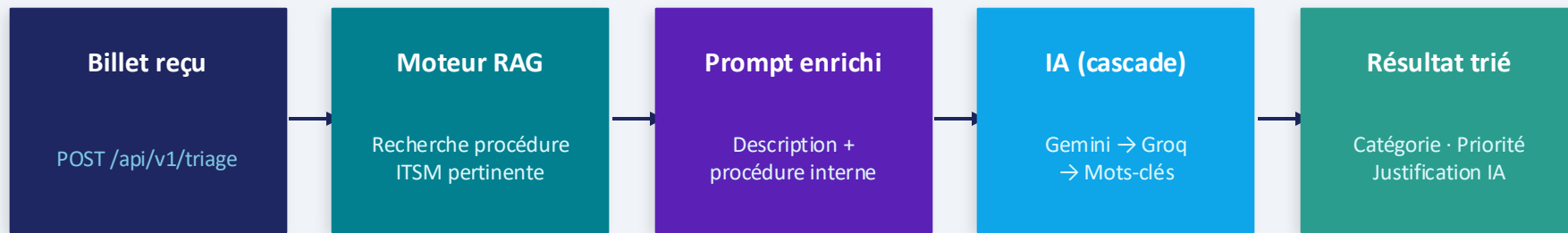
Bootstrap 5 + Jinja2

Interface web

Landing, portail demandeur, dashboard
technicien

+ NumPy (similarité cosinus) · python-dotenv · Pydantic · Hugging Face Inference API (fallback embeddings)

Architecture et flux de triage



Chaque étape est persistée atomiquement en SQLite : Ticket → RagHistory → TriageResult

Cascade de fallback IA — disponibilité garantie à 100 %

1

Gemini 2.5 Flash

Priorité principale — sortie JSON structurée avec Pydantic

2

Groq / Llama 3.3

Fallback automatique si Gemini retourne une erreur 429 (quota)

3

Algorithme mots-clés

Triage local pur, aucune dépendance réseau, actif si les deux IA sont indisponibles

Moteur RAG — Retrieval-Augmented Generation

Le RAG ancre la réponse de l'IA dans des procédures ITSM internes réelles — éliminant les hallucinations et guidant le modèle vers une procédure pertinente avant tout classement.

TicketRAGBasic

En mémoire

- Embeddings Gemini vectorisés au démarrage
- Similarité cosinus avec NumPy
- Cache SHA-256 pour éviter les appels API inutiles
- Idéal : démo rapide, environnement sans ChromaDB

TicketRAGChroma

Persistant sur disque

- Base vectorielle ChromaDB (src/data/chroma_db/)
- Synchronisation automatique de la knowledge base
- Distance cosinus native — converti en score : $1 - \text{distance}$
- Scalable : prêt pour un volume élevé de procédures

Paramètres communs — Seuil de similarité : **0.68** · Modèle d'embedding : **gemini-embedding-2** · Fallback embeddings : **Hugging Face (all-MiniLM-L6-v2)**

Le basculement entre les deux moteurs se fait en temps réel depuis le dashboard, sans redémarrer Flask.

Architecture des données

Ticket

- id (PK)
- description (Text)
- statut
- date_creation (UTC)
- date_modification

TriageResult

- id (PK)
- ticket_id (FK → Ticket)
- categorie
- priorite
- justification
- modele_ia
- tokens_utilises

RagHistory

- id (PK)
- ticket_id (FK → Ticket)
- contexte_retrouve
- source ([Basic]/[Chroma])
- score_similarite
- categorie_reference
- priorite_reference

Flux atomique : Validation → Ticket.flush() → RAG → RagHistory → IA → TriageResult → commit() — rollback() si erreur.

Correction recommandée : créer le ticket avec statut='en_attente' → mettre à jour après succès de l'IA.

Interface web — Trois portails distincts

Landing /

Page d'accueil permettant de choisir son rôle : Demandeur ou Technicien. Point d'entrée unique de l'application.

- Deux cartes avec icônes SVG
- Navigation vers /portail ou /dashboard
- Navbar responsive Bootstrap 5

Portail /portail

Interface simplifiée pour les utilisateurs finaux. Sans aucune option technique exposée.

- Formulaire épuré (description uniquement)
- Confirmation avec numéro de ticket #ID
- Affichage catégorie + priorité colorée
- Toast d'erreur non-bloquant

Dashboard /dashboard

Interface avancée réservée aux techniciens ITSM, avec accès à tous les contrôles.

- Sélection du modèle IA (Gemini/Groq/Auto)
- Switch moteur RAG Basic ↔ ChromaDB
- Tableau complet avec badges de priorité
- Edge cases pré-chargés + reset admin

Mise en marche — Installation et démarrage

1 Cloner & environnement virtuel

```
git clone <repo>
cd veille-techno-1SH
python -m venv venv
venv\Scripts\activate # Windows
source venv/bin/activate # Mac/Linux
```

2 Dépendances & clés API

```
pip install -r requirements.txt

# Créer .env à la racine :
GEMINI_API_KEY=votre_cle_ici
GROQ_API_KEY=votre_cle_groq # optionnel
TICKETFLOW_ADMIN_RESET_TOKEN=jeton_local
```

3 Initialiser la base de données

```
$env:FLASK_APP = 'run.py' # PowerShell
flask db init
flask db migrate -m 'init'
flask db upgrade
# → instance/ticketflow.db créé
```

4 Lancer le serveur

```
python run.py

# Interfaces disponibles :
# http://127.0.0.1:5000/ ← Accueil
# http://127.0.0.1:5000/portail ← Demandeur
# http://127.0.0.1:5000/dashboard ← Technicien
```


Scénarios de démonstration

Cas de test intégrés dans le dashboard (edge cases)

VPN instable

Entrée : "Plusieurs utilisateurs ne peuvent plus se connecter au VPN depuis ce matin."

Attendu : Réseau / Élevé

RAG : *VPN instable ou impossible à connecter (0.91)*

Hors sujet

Entrée : "J'ai un bogue dans mon RegistrationsController en Ruby on Rails."

Attendu : Logiciel / Faible

RAG : *Aucune procédure (< seuil 0.68)*

Multi-problèmes

Entrée : "URGENT: Rover Samsung ne s'enrôle pas dans Intune + compte Octopus verrouillé + imprimante bourrée."

Attendu : Accès / Élevé

RAG : *Enrôlement Intune (0.81)*

Faux positif BSOD

Entrée : "Mon fond d'écran est tout bleu. Peur d'un écran bleu Windows, mais c'est juste mon image."

Attendu : Matériel / Faible

RAG : *Écran bleu Windows (0.74)*

Ce que TicketFlow démontre

Un système ITSM IA de bout en bout

- ✓ API REST robuste avec cascade de fallback garantissant 100 % de disponibilité
- ✓ Moteur RAG double (Basic + ChromaDB) basculable sans redémarrage
- ✓ Persistance SQLite atomique — chaque ticket est tracé de la soumission au résultat IA
- ✓ Interface séparée Demandeur / Technicien — UX adaptée à chaque rôle